Reviewer Pack — Trace-Norm Capacity Conjecture for Read-Twice BP Communicati…

Entry 0bd071f00973 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 10:06:37 UTC

Trace-Norm Capacity Conjecture for Read-Twice BP Communication Matrices

Entry ID: `0bd071f00973` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 10:06:37 UTC

1. Conjecture

Field A (mathematical branch): Schatten-1 (nuclear/trace) norm geometry of sign matrices in the Linial–Shraibman γ_2/discrepancy-lifting tradition **Field B** (complexity object): Read-twice oblivious branching programs (Borodin–Razborov–Smolensky model), measured via their balanced communication matrix

Statement:

> Let P be a read-twice oblivious branching program of size s on n input bits (n even) computing f_P:{0,1}^n→{±1}. For the canonical balanced bipartition into the first n/2 and last n/2 input variables, form the $2^{\{n/2\}\times 2}\{n/2\}$ sign-matrix M_P[x,y]=f_P(x,y) and let ‖M_P‖* *denote its nuclear (sum of singular values) norm. Define ρ(P):=log_2(‖M_P‖*/2^{n/2})*; we conjecture there is an absolute constant C≤4 such that ρ(P)≤C·log_2(s+1) for every read-twice oblivious BP P, while the trivial truth-table BP for IP_2 (which has size 2^n) attains ρ=n/4=Ω(n), so any read-twice BP computing IP_2 must have size ≥2^{n/(4C)}.

Rationale (proposer's reasoning):

> For read-once oblivious BPs the communication matrix has real rank ≤width(P), instantly giving ρ≤log width, but this rank argument breaks at the second read — exactly the obstruction the Borodin–Razborov–Smolensky 1993 program ran into. Lifting BP size to a Schatten-1 norm (rather than rank) is a natural relaxation that survives shared layer reuse, and a polynomial trace-norm bound would be a clean operator-theoretic lift of BP size, automatically converting the Lindsey-lemma discrepancy 2^{-n/2} of IP_2 into an exponential read-twice lower bound. Because nuclear norm is convex and submultiplicative under layer products, it is a plausible candidate where free-probability / spectral attempts on this problem repeatedly stalled.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b7697f732fe0d78b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds × n∈{8,10,12,14,16} × w∈{2,3,4} (450 random read-twice BPs), every instance must satisfy ρ(P) ≤ 4·log₂(s+1)+1, and the IP_2 truth-table BP must satisfy ρ ≥ n/4−0.5 for all tested n. Support requires 100% pass on both conditions.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- read-twice oblivious branching program lower bound communication matrix` `- nuclear norm sign matrix Linial Shraibman gamma_2 branching program` `- inner product IP_2 read-twice branching program size lower bound discrepancy`

Top relevant hits considered: - [http://arxiv.org/abs/0809.3137v1] The Frontiers of Nuclear Science, A Long Range Plan - [http://arxiv.org/abs/0809.2093v2] An approximation algorithm for approximation rank - [http://arxiv.org/abs/1901.06841v1] Consistency Examinations of Calculations of Nuclear Matrix Elements of Double-β Decay by QRPA

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
```

```
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0]*p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def nuclear_norm(M):
        U, _, Vt = gaussian_elimination(matrix_multiply(M, M))
        singular_values = [math.sqrt(U[i][i]*Vt[i][i]) for i in range(min(len(U), len(Vt)))]
        return sum(singular_values)

    n = random.choice([8, 10, 12, 14, 16])
    w = random.choice([2, 3, 4])
    s = 2**w
    IP_2_trivial = [[(-1)**(i^j) for j in range(n//2)] for i in range(n//2)]

    def f(x, y):
        return (-1)**((x & (1 << n//2 - 1)) ^ (y & (1 << n//2 - 1)))

    M = [[f(i, j) for j in range(2**(n//2))] for i in range(2**(n//2))]
    rho_P = math.log2(nuclear_norm(M) / 2**(n//2))

    return {
        "metric_name": "rho(P)",
        "metric_value": rho_P,
        "instances_tested": 1,
        "conjecture_holds": rho_P <= 4 * math.log2(s + 1) + 1,
        "counterexample": "" if rho_P <= 4 * math.log2(s + 1) + 1 else f"rho(P) = {rho_P} > 4 * lo
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho_P = sum(r["metric_value"] for r in results) / len(results)
    std_rho_P = math.sqrt(sum((r["metric_value"] - mean_rho_P)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho_P} std={std_rho_P} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
```

```
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['cour
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47d72295.py", line 78, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47d72295.py", line 62, in run_trial
    rho_P = math.log2(nuclear_norm(M) / 2**(n//2))
                      ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47d72295.py", line 49, in nuclear_norm
    U, _, Vt = gaussian_elimination(matrix_multiply(M, M))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47d72295.py", line 31, in gaussian_elim
    A[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a ZeroDivisionError in a hand-rolled gaussian_elimination routine used to compute the nuclear norm, so no data was produced for either the random BP instances or the IP_2 baseline. Neither side of the pre-registered support condition could be evaluated. | next: Replace the buggy custom SVD with numpy.linalg.svd (nuclear norm = sum of singular values) and re-run the 450 random instances plus the IP_2 truth-table baseline across n∈{8,10,12,14,16}.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	189920
2	preregistration	claude_max	opus	0	0	8313
3	novelty	claude_max	opus	0	0	3762
4	novelty	claude_max	opus	0	0	7700
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15088

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16490
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14893
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10604
9	judge	claude_max	opus	0	0	4534

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 271303 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/0bd071f00973.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0bd071f00973.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0bd071f00973.tar.gz` (if generated)